

The Zen of Python

The Zen of Python by Tim Peters are 20 guidelines for the design of the Python language.

Your Python code doesn't necessarily have to follow these guidelines, but they're good to keep in mind.

The Zen of Python is an *Easter egg*, or hidden joke, that appears if you run `import this`:

```
import this
```

```
...
```

```
The Zen of Python, by Tim Peters
```

```
Beautiful is better than ugly.
```

```
Explicit is better than implicit.
```

```
Simple is better than complex.
```

```
Complex is better than complicated.
```

```
Flat is better than nested.
```

```
Sparse is better than dense.
```

```
Readability counts.
```

```
Special cases aren't special enough to break the rules.
```

```
Although practicality beats purity.
```

```
Errors should never pass silently.
```

```
Unless explicitly silenced.
```

```
In the face of ambiguity, refuse the temptation to guess.
```

```
There should be one-- and preferably only one --obvious way to do it.
```

```
Although that way may not be obvious at first unless you're Dutch.
```

```
Now is better than never.
```

```
Although never is often better than *right* now.
```

```
If the implementation is hard to explain, it's a bad idea.
```

```
If the implementation is easy to explain, it may be a good idea.
```

```
Namespaces are one honking great idea -- let's do more of those!
```

```
...
```

NOTE: Mysteriously, only 19 of the guidelines are written down. Guido van Rosum reportedly said that the missing 20th aphorism is “some bizarre Tim Peters in-joke.”

In the end, these guidelines are opinions that can be argued for or against. Like all good sets of moral codes, they sometimes contradict themselves to provide the most flexibility. Here’s a interpretation of these aphorisms:

Beautiful is better than ugly.

Programmers often write code quickly without concern for readability. While code doesn’t have to be readable, the code of the Python language itself must be thought out, consistent, and a joy to use. Of course, not every script needs to be beautiful, and beauty is subjective, but much of Python’s popularity is a result of being so easy to work with.

Explicit is better than implicit.

“This aphorism is self-explanatory,” that would be a terrible explanation for any aphorism. Similarly, it’s better for code to be verbose and explicit. You should avoid hiding code’s functionality behind obscure language features that require familiarity to fully understand.

Simple is better than complex. Complex is better than complicated.

These two aphorisms remind us that building anything can be done using simple or complex techniques. With a simple problem, such as building a birdhouse, a simple solution is better. Building a diesel train engine, on the other hand, is a complex problem that requires complex techniques. Even if you could technically make a diesel train engine using birdhouse techniques, you probably would end up with a complicated, Rube Goldberg arrangement of birdhouse parts that wouldn’t be an ideal solution. Prefer simplicity to complexity, but know the limits of simplicity.

Flat is better than nested.

Programmers love to organize things into categories, especially categories that contain subcategories which contain other sub-subcategories. These hierarchies often don’t add *organization* so much as they add *bureaucracy*. It’s okay to have code in just one top-layer module or class instead of split up across multiple submodules or subclasses. If you make packages and modules that require code like `import spam.eggs.bacon.ham.foo.bar`, then you’re making your code too complicated.

Sparse is better than dense.

Programmers often like to cram as much functionality into as little code as possible, such as one-liners like the following: `print('\n'.join("%i bytes = %i bits which has %i possible values." % (j, j*8, 256**j-1) for j in (1 << i for i in range(8))))`. While code like this may impress their friends, it’ll infuriate their coworkers who have to try to understand it. Code that is spread out over multiple lines is often easier to read than dense one-liners.

Readability counts.

While `strcmp()` may obviously mean the “string compare” function to someone who has been programming in C since the 1970s, modern computers have enough memory to write out the full function name. Don’t drop vowels from your names or write overly terse code. Code is read more often than it’s written, so explicit, readable code is more important than terse, undocumented code.

Special cases aren’t special enough to break the rules. Although practicality beats purity.

These two aphorisms, which come as a set, contradict each other. Programming is full of “best practices” that programmers should strive for in their code. Skirting these practices for a quick hack may be tempting, but can lead to a rat’s nest of inconsistent, unreadable code. However, bending over backwards to adhere to rules can result in highly-abstract, unreadable code. The Java programming language’s attempt to fit all code to its object-oriented paradigm often results in lots of boilerplate code for even the smallest program. Walking the line between these two aphorisms becomes easier with experience. And in time, you’ll not only learn the rules, but also learn when to break them.

Errors should never pass silently. Unless explicitly silenced.

Just because programmers often ignore error messages doesn’t mean the program should stop emitting them. Silent errors can happen when functions return error codes or `None` instead of raising exceptions. These two aphorisms tell us that it’s better for a program to fail fast and crash than to silence the error and continue running the program. The bugs that inevitably happen later on will be harder to debug since they are far removed from the original cause. Though you can always choose to explicitly ignore the errors your programs cause, just be sure you are making the conscious choice to do so.

In the face of ambiguity, refuse the temptation to guess.

Computers have made humans superstitious: To exorcise the demons in our computers we perform the sacred ritual of turning them off and then on. Supposedly this will fix any mysterious problem. However, computers are not magic. If your code isn’t working, there is a reason and only careful, critical thinking will solve it. Refuse the temptation to blindly try solutions until something seems to work; often you have merely masked the problem rather than solved it.

There should be one—and preferably only one—obvious way to do it.

This is a broadside against the Perl programming language’s motto, “There’s more than one way to do it!” It turns out that having three or four different ways to write code that does the same thing is a double-edged sword: you have flexibility in how you write code, but now you have to learn every possible way it could have been written in order to read it. This flexibility isn’t worth the 3x or 4x effort needed to learn a programming language.

Although that way may not be obvious at first unless you're Dutch.

This line is a joke. Guido van Rossum, the creator and *BDFL* (Benevolent Dictator for Life) of Python, is Dutch. However, not even this aphorism prevented Python from incorporating three different ways of formatting strings.

Now is better than never. Although never is often better than **right now.**

These two aphorisms tell us that code that hangs or gets caught in infinite loops is obviously worse than code that doesn't. However, it's almost certainly better to wait for your program to finish than to have it finish too early with incorrect results.

If the implementation is hard to explain, it's a bad idea. If the implementation is easy to explain, it may be a good idea.

Python strives to make the programmer's job easier rather than accommodate the computer so a program runs faster. And programs need to be understandable not just by the programmer who wrote it, but also by other programmers who maintain the code. These two aphorisms remind us that if "high-performance" code is so complicated as to be impossible for programmers to understand and debug, then it's bad code. But alas, just because it's easy to explain a program's code to someone else doesn't mean it isn't bad code. Programming is hard.

Namespaces are one honking great idea—let's do more of those!

Namespaces (and also global and local scopes) are key for preventing names in one module or scope from conflicting with names in another. But also remember that *flat is better than nested*: As great as they are, namespaces should be made only to prevent naming conflicts, and not to add needless categorization.